# PCS-302 : Data Structures Lab

**Boolean Matrix | Row with Maximum Number of 1's (Linear Time)**

## Problem Statement

Given a Boolean matrix (containing only 0 and 1) of size m × n, where each row is sorted in increasing order, design an algorithm and write a program to find which row has the maximum number of 1's. The solution must run in linear time.

## Example

```
Boolean matrix (4x5), each row sorted increasing (0's then 1's):

Row 0:  0  0  0  1  1     (2 ones)
Row 1:  0  1  1  1  1     (4 ones)
Row 2:  0  0  1  1  1     (3 ones)
Row 3:  0  0  0  0  1     (1 one)

Row with maximum 1's  -->  Row 1
```

## Algorithm (Staircase Search)

```
1. col = n - 1, maxRow = -1

2. For row = 0 to m - 1:
       While col >= 0 AND mat[row][col] == 1:
             col = col - 1
             maxRow = row          // this row beats the previous best

3. Return maxRow

Why linear: the col pointer only ever moves left and is
never reset, so across all rows it moves at most n times
in total. Combined with m rows, total work = O(m + n).
```

## C Program

```
#include <stdio.h>
#define M 4
#define N 5

int main() {
    int mat[M][N] = {
        {0, 0, 0, 1, 1},
        {0, 1, 1, 1, 1},
        {0, 0, 1, 1, 1},
        {0, 0, 0, 0, 1}
    };

    int row, col;
    int maxRow = -1;      // row with maximum 1's so far
    col = N - 1;           // start at the last column
    for (row = 0; row < M; row++) {
```

```
        for ( ; col >= 0 && mat[row][col] == 1; col--) {
            maxRow = row;   // this row has more 1's than the previous best
        }
    }

    if (maxRow == -1)
        printf("No row contains a 1.\n");
    else
        printf("Row %d has the maximum number of 1's.\n", maxRow);

    return 0;
}
```